Enigma of Life
Technical Rework and Architecture Report

Conway Puzzle Game in Pygame

Technical rework by Rafael Maestre López

CentraleSupelec

April 11, 2026

Team Presentation



Rafael Maestre López



Nils Malpot



Gabriel Sylvestre-Nunez



Louis Francois



Hugo Charles



Romain Chambon

Outline

Scope and Intent
System Architecture
Mathematical Formulation
File-by-File Technical Explanation
Engineering Notes

1 Scope and Intent

2 System Architecture

3 Mathematical Formulation

4 File-by-File Technical Explanation

5 Engineering Notes

Report Scope

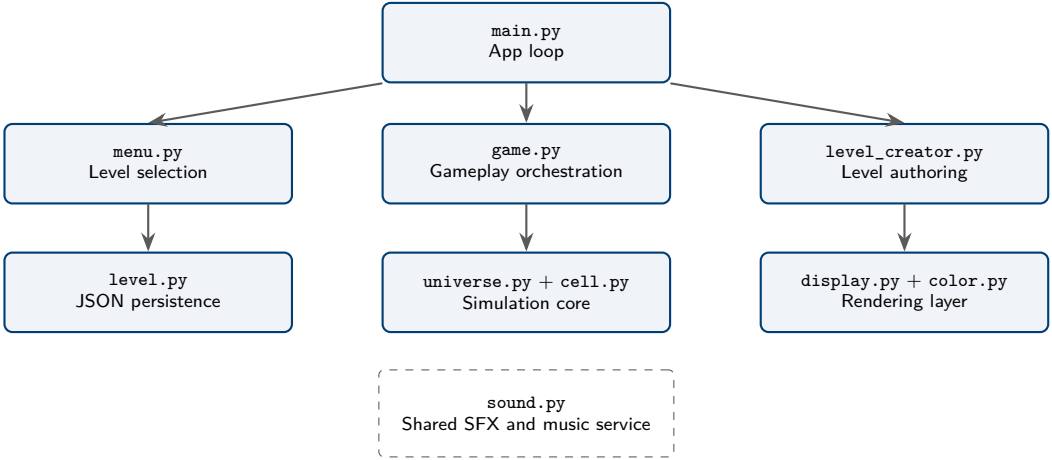
Purpose

- Provide a technical explanation of how each source file works.
- Explain data flow, runtime control flow, and module coupling.
- Document the mathematical model behind gameplay behavior.

Authoring statement

- This document describes a technical rework performed by Rafael Maestre López.
- Focus is engineering structure and implementation logic, not project chronology.

Module Architecture



Primary dependencies are shown explicitly for readability.
Cross-cutting reuse omitted from the graph: `display.py` and `sound.py` are shared by menu, gameplay, and creator; `level.py` and `universe.py` are also reused across multiple controllers.

Global Execution Pipeline


```
pygame.init()
screen = pygame.display.set_mode((W, H))
sounds = SoundManager()

while True:
    choice = Menu(screen, sounds).run()
    if choice in (None, "quit"):
        break
    if choice == "creator":
        LevelCreator(screen, sounds).run()
    if choice.startswith("level:"):
        Game(screen, sounds).play_level(level_name)

pygame.quit()
```

The runtime is mode-based: menu, creator, and gameplay are independent loops coordinated by `main.py`.

Cellular Automaton Dynamics

Scope and Intent
System Architecture
Mathematical Formulation
File-by-File Technical Explanation
Engineering Notes

For each non-permanent, non-objective cell at position (r, c):

$$A_{t+1}(r,c) = \begin{cases} 1 & \text{if } A_t(r,c) = 1 \wedge N_t(r,c) \in \{2,3\} \\ 1 & \text{if } A_t(r,c) = 0 \wedge N_t(r,c) = 3 \\ 0 & \text{otherwise} \end{cases}$$

where $N_t(r,c)$ is the count of alive neighbors.

Toroidal indexing used in `universe.py`:

$$(r', c') = (r \bmod R,\ c \bmod C)$$

This removes boundary-condition branching and guarantees uniform local rules.

Objective Cells and Win Criterion

Objective evolution rule

- Objective cells behave as dead cells until activated.
- If exactly three alive neighbors exist, objective state becomes alive.

Win tracking

$$\mathcal{R}_{t+1} = \mathcal{R}_t \cup \{(r, c) \in \mathcal{O} \mid A_{t+1}(r, c) = 1\}$$

Level clear condition:

$$|\mathcal{R}_t| = |\mathcal{O}|$$

where $\mathcal{O}$ is the objective set and $\mathcal{R}$ the reached-objective set.

This cumulative criterion prevents loss of progress when an already reached objective later dies.

Input Mapping and Time Stepping

Mouse position to grid index (used in `game.py` and `level_creator.py`):

$$\text{col} = \left\lfloor \frac{m_x - o_x}{s} \right\rfloor, \qquad \text{row} = \left\lfloor \frac{m_y - o_y}{s} \right\rfloor$$

with cell size $s =$ `CELL_SIZE` and board offset (o_x, o_y).

Simulation timing:

$$\Delta t \leftarrow \texttt{clock.tick(FPS)}, \qquad \text{step when accumulated } \Delta t \geq \texttt{SIMULATION_DELAY}$$

This decouples rendering frame rate from simulation generation rate.

Configuration and Visual Tokens

`constants.py`

- Global runtime constants: window size, grid size, FPS, simulation delay.
- Asset paths for font and sound directories.
- Font fallback strategy (file font first, system font fallback).

`color.py`

- Single source of truth for retro palette.
- Separates semantic tokens (e.g. `OBJECTIVE`, `PANEL_BG`) from rendering code.
- Enables deterministic visual identity across menu, gameplay, and editor.

Cell and Universe Core Logic

`cell.py`

- Defines discrete states: dead, alive, permanent, objective.
- Exposes semantic properties: `is_alive`, `is_permanent`, `is_objective`.

`universe.py`

- Owns the 2D grid and generation transitions.
- Implements wrapped neighbor lookup and generation update.
- Implements objective-cell activation behavior.
- Utility methods for copying and objective checks.

Complexity per generation is $\mathcal{O}(R \times C)$.

Level Persistence Layer

`level.py`

- Data transfer object for level metadata and grid seeds.
- JSON serialization/deserialization API:
  - `save()` writes structured schema to `levels/`
  - `load(name)` reconstructs runtime level object
- Level-to-universe materialization via `build_universe()`.

Current schema includes: `name`, `rows`, `cols`, `permanents`, `alive_cells`, `objectives`, `start_zone`, `music_track`.

Rendering Engine

`display.py` provides reusable rendering primitives:

- Background pipeline: gradient + starfield + scanline overlay.
- Panel and button drawing with retro bevel effects.
- Universe rendering: state-aware tiles, objective markers, start-zone overlay.
- Text rendering with shadow and font caching.

Architectural role:

- Centralizes drawing logic so state modules remain UI-agnostic.
- Keeps menu/game/editor visual language consistent.

Audio Engine

`sound.py`

- Initializes mixer and handles failure gracefully (audio optional).
- Resolves SFX file formats in priority order:

$$\texttt{.wav} \rightarrow \texttt{.ogg} \rightarrow \texttt{.mp3}$$

- Loads semantic SFX map (UI, placement, simulation controls, win).
- Manages streaming background music per level (start/stop, dedup by track).

The interface is intentionally small: `play()`, `play_music()`, `stop_music()`, `missing_files()`.

UI Controller: Main Menu

`menu.py`

- Collects available level files dynamically from persistence layer.
- Handles interaction events and returns explicit routing commands:
  - `level:<name>`
  - `creator`
  - `quit`
- Delegates all painting to `display.py`, all audio feedback to `sound.py`.

Result: menu has high cohesion (state + events) and low coupling (render/audio are services).

UI Controller: Gameplay Runtime

`game.py`

- Integrates all core services during level execution.
- Responsibilities:
  - load level and start track
  - place initial live cells in start zone
  - advance simulation clock and detect win condition
  - render board/HUD/win overlay
  - guarantee music stop on exit using `try/finally`
- Owns per-run mutable state: `placing`, `simulating`, `paused`, `won`, and reached-objective set.

UI Controller: Level Creator

`level_creator.py`

- In-editor universe sandbox for authoring new puzzle maps.
- Tools: permanent, alive, objective, start-zone rectangle.
- Text-box based level naming with focus management.
- Serializes generated level through `level.py`.

Internal invariant:

- Tracking arrays (`permanents`, `objectives`) are kept synchronized with grid state through toggle/remove logic.

Entry Point and Mode Routing

`main.py`

- Creates Pygame context and display surface.
- Creates shared `SoundManager` once and injects it into all modes.
- Implements top-level routing loop between menu, gameplay, and creator.

This file intentionally contains almost no business logic; it is a coordinator and lifecycle manager.

Original Contributions by Rafael Maestre López

Gameplay control and analysis upgrades in `game.py`

- Predictive setup overlay (key `P`): computes $U_{t+1} = F(U_t)$ and highlights births/deaths before starting simulation.
- Bounded undo/redo (keys `Z`/`Y`): stores setup deltas (r, c, s_{old}, s_{new}) in finite history stacks for deterministic editing.
- Multi-tempo simulation (key `T`): cycles speed factors $k \in \{1, 2, 4\}$ with effective step interval

$$\Delta_{\text{step}} = \max\left(40, \left\lfloor \frac{\Delta_0}{k} \right\rfloor\right)$$

- Paused single-step execution (key `N`): advances exactly one generation for pattern debugging and puzzle strategy planning.

Design Decisions and Tradeoffs

- **Toroidal topology**: simple and deterministic, but not physically bounded.
- **Cumulative objective progress**: better puzzle UX than instantaneous check.
- **Service separation**: rendering and audio are utility layers, reducing duplication.
- **JSON persistence**: easy modding and debugging, weak schema guarantees without validation.

Overall strategy: prefer explicit, maintainable code paths over highly abstract architecture.

Extension Points

1. Add formal schema validation for level JSON files.
2. Add automated tests for:
   - generation transitions
   - objective activation and completion
   - creator save/load round-trip
3. Add configurable boundary modes (toroidal vs bounded).
4. Expose music-track selection directly in creator UI.

Conclusion

- The project is organized as a clear layered system: state logic, persistence, rendering, audio, and mode controllers.
- Core mechanics are mathematically well-defined and implemented consistently across runtime modules.
- The current codebase is stable for both gameplay use and further technical evolution.

Enigma of Life now has a documented technical architecture with explicit file-level responsibilities.